<u>Algorithm to convert an infix expression to postfix form</u>.

Start:

s → the empty stack.

char infix[100] → the input infix expression

char postfix[100] → the output postfix expression

symbol → a character variable to represent each character from infix string.

t → a temporary variable to store the popped character

```
while(until we reach the last character of infix string)
{
symbol = next input character

if (symbol is an operand)
   add symbol to postfix string
else
    if( symbol is '(' )
         push(symbol);
    else
         if( symbol is ')' )
           {
             while((t=pop()) != '(' )
               {
                  add the popped character t to the postfix string.
               }
           }
          else
           {
              while((the stack is not empty) && (precedence(top of the stack)>=
                                                     precedence(symbol)))
                   {
                      t=pop();
                      add the popped character t to the postfix string.
                    }
                   push(symbol);
             } /* close else */
 } /* close while */
```

```
 while(stack is not empty())
    {
      t=pop();
      add popped character t to the postfix string.
    }
print the postfix string.

End.
```